Topological properties of a 2-dimensional torus-connected cycles network

Antoine Bossard
Graduate School of Science, Kanagawa University
Hiratsuka, Kanagawa, Japan

ABSTRACT

Modern supercomputers are massively parallel systems: they include hundreds of thousands of compute nodes. Conventional topologies previously used for the interconnection network of these systems, like the hypercube topology, are no longer suitable as they would induce an overly large number of links, in other words an excessively high network degree. Hence, advanced network topologies have been proposed for this purpose. Hierarchical interconnection networks, and especially torus-based ones have proven very popular: several highly ranked supercomputers, such as the world no. 1 as of May 2022, the Fujitsu Fugaku, have been relying on it. The torus-connected cycles topology (TCC) has been introduced after recognising this trend of the supercomputing industry. Several topological properties of TCCs have been shown; for instance, it is known that a TCC is hamiltonian. In this paper, we further focus on showing such topological properties of TCCs. Precisely, we consider in a 2-dimensional k-ary TCC the symmetry property, important for algorithm design, and the crossing number property, which has applications in circuit design and graph visualisation. On the one hand, a 2-dimensional k-ary TCC is shown to be not symmetric, and on the other hand, an upper bound on its crossing number is established. The obtained upper bound is then evaluated by means of a computer experiment.

CCS CONCEPTS

• **Computer systems organization** → **Parallel architectures**; **Dependable and fault-tolerant systems and networks**; • **Mathematics of computing** → **Graph theory**; • **Hardware** → *Very large scale integration design.*

KEYWORDS

TCC, mesh, interconnect, network, graph

ACM Reference Format:
Antoine Bossard. 2022. Topological properties of a 2-dimensional torus-connected cycles network. In *The 10th International Conference on Computer and Communications Management (ICCCM 2022), July 29–31, 2022, Okayama, Japan.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3556223.3556244

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICCCM 2022, July 29–31, 2022, Okayama, Japan
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9634-9/22/07…$15.00
https://doi.org/10.1145/3556223.3556244

1 INTRODUCTION

As shown by the recent world rankings of TOP500, the torus topology has proven very popular for the interconnect of modern supercomputers. For instance, the IBM BlueGene/Q is based on a 5-dimensional torus [9] and the Fujitsu K and the Fujitsu Fugaku (a.k.a. Post-K computer) – the world no. 1 supercomputer as of May 2022 [13] – both on a 3-dimensional torus (the Tofu interconnect) [1, 2].

In accordance with these examples, the network dimension usually stays rather low. So, it first makes sense to consider a hierarchical interconnection network (HIN [6]) based on the torus topology, just like the Tofu interconnect, as it enables to increase the network order (i.e. the number of nodes) while retaining the network diameter (i.e. the maximum distance between any two nodes) low. For example, the Tofu interconnect combines a 3-dimensional torus on the higher level with a 3-dimensional (2, 3)-ary torus on the lower level [2] (formal topology definitions are given in the next section).

Then, it is meaningful to consider a low network dimension to interconnect nodes: the network degree (i.e. the number of links per node) thus remains low. This low degree property is critical as it is well known that physical constraints severely limit the tolerable maximum number of links per node. To reuse the previously given examples, the interconnect of the IBM BlueGene/Q supercomputer features ten links per node, just as the Tofu interconnect of the Fujitsu K supercomputer [2].

Now, if we consider the number of nodes embedded inside modern supercomputers, also appropriately referred to as massively parallel systems, the pertinence of HINs becomes clear. For example, the hypercube interconnection network as used in the early days of supercomputing [12] would under such constraints (i.e. ten links per node) be able to connect only $2^{10} = 1\,024$ nodes. This is to be compared to, for instance, the 10 649 600 nodes of the Sunway TaihuLight supercomputer [14].

The torus-connected cycles topology (TCC) has been proposed as interconnection network of massively parallel systems considering this trend of the supercomputing industry [5]. Several properties of a TCC network have been previously established, for instance that it is hamiltonian [7]. In this paper, we discuss additional topological properties of a 2-dimensional k-ary TCC network: the symmetry property and the crossing number property. These two properties have important applications such as algorithm design for the former and circuit design (as for VLSI) and graph visualisation for the latter. Besides, as mentioned previously, modern supercomputers based on a torus usually retain the network dimension low, hardware and software simplicity, for instance for routing algorithms, being certainly key. Therefore, considering a 2-dimensional TCC is in this regard definitely relevant.

2 PRELIMINARIES

We first recall in this section various definitions of graph theory. Unless stated otherwise, graph theory notations and definitions are as per [10]. A graph is made of vertices (a.k.a. nodes) and edges (a.k.a. links); an edge connects two distinct vertices. An arc is a directed edge: formally, for any two adjacent vertices u, v, whereas the two edges $\{u, v\}$ and $\{v, u\}$ are equal, the two arcs (u, v) and (v, u) are not. The edges of a directed graph are arcs.

Definition 2.1. [4] A graph $G = (V, E)$ is symmetric if and only if for any two arcs (u_1, v_1), (u_2, v_2) there exists an automorphism α of G that satisfies $\alpha(u_1) = u_2$ and $\alpha(v_1) = v_2$.

A symmetric graph is also called arc-transitive (1-arc-transitive) and flag-transitive [3].

Definition 2.2. [8] When drawing a graph G on the plane, the minimum number of edge crossings is called the *crossing number* of the graph; it is denoted by $cr(G)$.

Definition 2.3. [11] An n-dimensional mesh has k_i vertices on the i-th dimension, where $k_i \geq 2$ and $0 \leq i \leq n-1$. Hence, an n-dimensional $(k_0, k_1, \ldots, k_{n-1})$-ary mesh includes $k_0 \cdot k_1 \cdot \ldots \cdot k_{n-1}$ vertices in total. A vertex u is addressed with n coordinates $(u_0, u_1, \ldots, u_{n-1})$. Two vertices u, v are adjacent if and only if $u_i = v_i$ holds for all i, $0 \leq i \leq n-1$, except one, j, where either $u_j = v_j + 1$ or $u_j = v_j - 1$.

Definition 2.4. [11] An n-dimensional torus of arity k, also called (n, k)-torus, is an n-dimensional $(k_0, k_1, \ldots, k_{n-1})$-ary mesh with $k_i = k$ for all i, $0 \leq i \leq n-1$, and with additional edges: two vertices u, v are adjacent if and only if $u_i = v_i$ holds for all i, $0 \leq i \leq n-1$, except one, j, where either $u_j = v_j + 1 \pmod k$ or $u_j = v_j - 1 \pmod k$ holds. These edges not present in the underlying n-dimensional mesh are called *wrap-around* edges.

The torus-connected cycles topology has been introduced as a HIN: it consists of two layers. The lower layer connects vertices according to the ring topology [11], inducing what we call *clusters*, and the higher layer connects vertices of distinct clusters according to the torus topology (see Definition 2.4). Its formal definition is recalled below.

Definition 2.5. [5, 7] An n-dimensional k-ary torus-connected cycles network TCC(n, k) is an undirected graph that consists of $2nk^n$ vertices. Each vertex a is a pair $(c(a), p(a))$ with $c(a) = (a_0, a_1, \ldots, a_{n-1})$ the cluster ID and $p(a) = p_a$ the processor ID, where $0 \leq a_i \leq k-1$ and $0 \leq p_a \leq 2n-1$. Each vertex a has three adjacent vertices $n_1(a)$, $n_2(a)$ and $n_3(a)$ as follows:

$$
\begin{aligned}
n_1(a) &= (c(a), (p_a + (-1)^{p_a}) \bmod 2n) \\
n_2(a) &= (c(a), (p_a - (-1)^{p_a}) \bmod 2n) \\
n_3(a) &= ((a_0, a_1, \ldots, a_{\lfloor p_a/2 \rfloor - 1}, (a_{\lfloor p_a/2 \rfloor} + (-1)^{p_a}) \bmod k, \\
&\qquad a_{\lfloor p_a/2 \rfloor + 1}, \ldots, a_{n-1}), p_a + (-1)^{p_a})
\end{aligned}
$$

For the sake of simplicity, the address of a TCC(n, k) node $a = ((a_0, a_1, \ldots, a_{n-1}), p_a)$ is denoted more concisely by the vector $(a_0, a_1, \ldots, a_{n-1}, p_a)$.

The edges that connect vertices of a same cluster are called *cluster edges* and those that connect vertices of distinct clusters are called *torus edges*. Just as in a torus, the torus edges of a TCC include wrap-around edges. Since only two dimensions are considered here, the wrap-around edges can be called either horizontal or vertical. A TCC$(2, 3)$ is illustrated in Figure 1. Wrap-around edges are cut for clarity: each is labelled with a letter for identification. Precisely, the three horizontal wrap-around edges are labelled D, E and F, and the three vertical wrap-around edges A, B, and C.

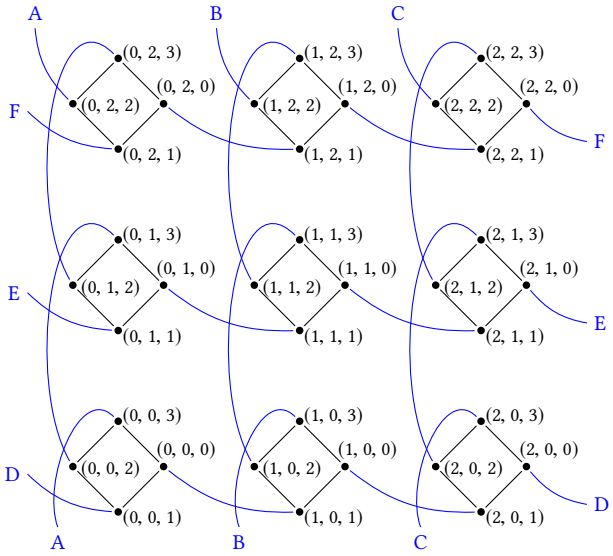


Figure 1: An illustration of a 2-dimensional 3-ary torus-connected cycles network TCC$(2, 3)$**. Torus edges are drawn in blue and cluster edges in black. Wrap-around edges are cut for the sake of clarity: each is labelled with a letter for identification.**

3 ON SYMMETRY IN A 2-DIMENSIONAL k-ARY TCC

As recalled in Section 2, a TCC(n, k) has a hierarchical structure which combines rings according to the torus topology – precisely, each vertex of an (n, k)-torus is replaced by a $2n$-ring. Both a ring and a torus are symmetric graphs [11]. Although it is thus perhaps slightly counterintuitive, a 2-dimensional k-ary TCC is not symmetric as shown in the following proposition.

Proposition 3.1. *A 2-dimensional k-ary torus-connected cycles network* TCC$(2, k)$ *is not symmetric.*

Proof. Consider the two arcs $e_1 = ((0, 0, 0), (0, 0, 1))$ and $e_2 = ((0, 0, 0), (1, 0, 1))$ of a TCC$(2, k)$. We show that for this graph there does not exist an automorphism that maps the arc e_1 to the arc e_2. Refer to Figure 2.

We proceed by contradiction: we assume that there exists such an automorphism that maps e_1 to e_2. Let it be called α. Then, because the two vertices $(0, 0, 1)$ and $(0, 0, 2)$ are adjacent, the vertex $\alpha((0, 0, 2))$ is adjacent to $\alpha((0, 0, 1))$ by definition of α. Hence, by definition of a TCC$(2, k)$, there are only two mapping candidates for $\alpha((0, 0, 2))$: either $\alpha((0, 0, 2)) = (1, 0, 2)$ or $\alpha((0, 0, 2)) = (1, 0, 0)$.

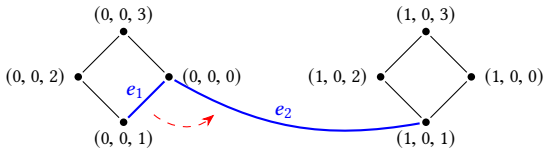


Figure 2: A part of a TCC$(2, k)$ to illustrate the arc mapping from $e_1 = ((0, 0, 0), (0, 0, 1))$ to $e_2 = ((0, 0, 0), (1, 0, 1))$.

On the other hand, because the vertex $(0, 0, 3)$ is adjacent to both $(0, 0, 0)$ and $(0, 0, 2)$, the vertex $\alpha((0, 0, 3))$ is adjacent to $\alpha((0, 0, 0)) = (0, 0, 0)$ and $\alpha((0, 0, 2))$. However, by definition of a TCC$(2, k)$, there exists no vertex that is adjacent to both $\alpha((0, 0, 0)) = (0, 0, 0)$ and one of $\alpha((0, 0, 2)) = (1, 0, 2)$ and $\alpha((0, 0, 2)) = (1, 0, 0)$. This is thus a contradiction: there exists no such vertex $\alpha((0, 0, 3))$.

Therefore, there exists no automorphism that maps the arc e_1 to the arc e_2, which suffices to show that a TCC$(2, k)$ is not symmetric. □

4 AN UPPER BOUND ON THE CROSSING NUMBER OF A 2-DIMENSIONAL k-ARY TCC

We establish in this section an upper bound on $cr(\text{TCC}(2, k))$ the crossing number of a 2-dimensional k-ary TCC when it is drawn on the plane. Establishing exactly the crossing number of a graph is a difficult, NP-hard problem. We proceed in several steps to show the difficulty of the task and how improvements are made. Although the figures given herein illustrate the case $k = 3$ (i.e. TCC$(2, 3)$) for the sake of clarity, the described drawing methods are obviously applicable to the general case (i.e. to a TCC$(2, k)$). The obtained upper bounds on the crossing number are thus given for the general case (i.e. on $cr(\text{TCC}(2, k))$).

4.1 Original drawing method of a TCC$(2, k)$

When the torus-connected cycles topology was originally introduced, it was imagined, and illustrated on a par with the drawing shown in Figure 3 [5, 7].

This drawing is sub-optimal with respect to the number of crossings. Precisely, although the cluster edges induce no crossing, we can see from Figure 3 that, informally, the torus edges are responsible for two crossings at each cluster (highlighted in blue in this figure) and, at other locations, the special torus edges that we called wrap-around edges induce as many crossings as the number of elements of the Cartesian product of the horizontal and vertical wrap-around edges (highlighted in red in this figure), that is k^2 crossings. So, in total, this drawing of a TCC$(2, k)$ induces $2k^2 + k^2 = 3k^2$ crossings since there are k^2 clusters. Figure 3 illustrates the case $k = 3$, with thus a total of 27 crossings.

4.2 First improvement

As a first improvement, we reorder the drawing of clusters so that the number of crossings induced by the torus edges at each cluster (highlighted in blue in Figure 3) is decreased from two to one. See Figure 4. Although the network looks differently when compared to

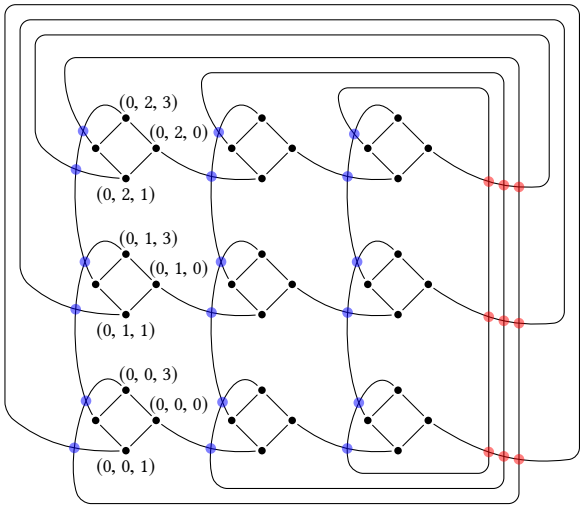


Figure 3: Original drawing method of a TCC$(2, k)$: in total, $3k^2$ crossings are induced (highlighted in blue and red). This figure illustrates the case $k = 3$, with thus a total of 27 crossings.

that of Figure 3, the two graphs are in fact isomorphic (automorphic) as shown with the addresses given in both figures.

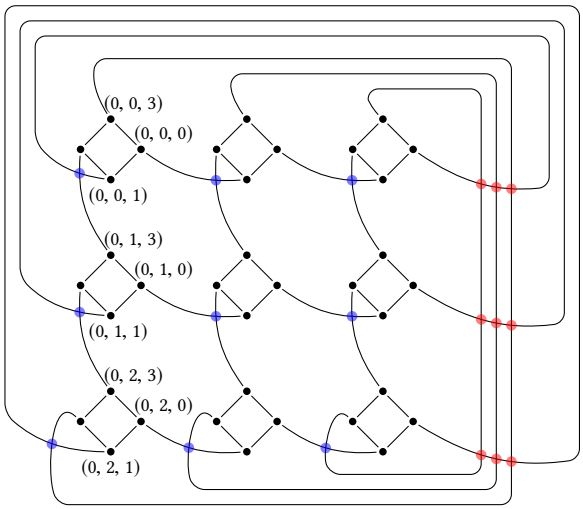


Figure 4: First improvement of the TCC drawing method: the total number of crossings is reduced from $3k^2$ to $2k^2$ (highlighted in blue and red). This figure illustrates the case $k = 3$, with thus a total of 18 crossings.

Therefore, this new drawing method induces k^2 crossings at each cluster (highlighted in blue) and k^2 crossings by the wrap-around edges on their own at other locations (highlighted in red). This improvement thus allows to reduce the total number of crossings from $3k^2$ to $2k^2$. Figure 4 illustrates the case $k = 3$, with thus a total of 18 crossings. This drawing method can be further refined as shown in the next section.

4.3 Second improvement

This second improvement focuses on reducing the number of the crossings that involve two wrap-around edges, especially the crossings highlighted in red in Figure 4. To this end, the k vertical wrap-around edges are drawn so that each crosses only $k-1$ horizontal wrap-around edges, and that once, instead of k previously, with one being crossed twice. See Figure 5; the vertical wrap-around edges are highlighted in green.

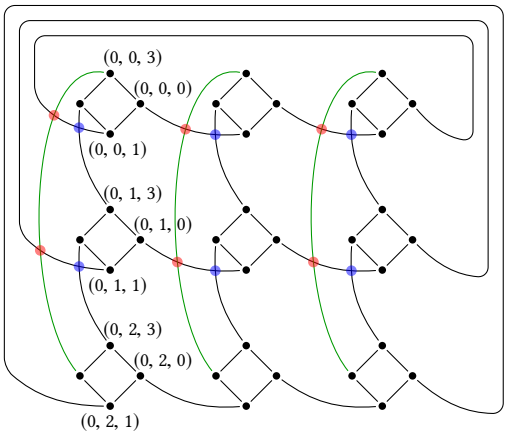


Figure 5: Second improvement of the TCC drawing method: the total number of crossings is further reduced from $2k^2$ to $2k^2-2k$ (highlighted in blue and red). This figure illustrates the case $k=3$, with thus a total of 12 crossings.

Therefore, this new drawing method induces $k(k-1)$ crossings at each cluster (highlighted in blue) and $k(k-1)$ crossings by the vertical wrap-around edges (highlighted in red). This improvement thus allows to reduce the total number of crossings from $2k^2$ to $2k^2-2k$. Figure 5 illustrates the case $k=3$, with thus a total of 12 crossings. We further refine this drawing method in the next section.

4.4 Third improvement

The third improvement concerns refining the drawing method of the topmost horizontal wrap-around edge so that it induces one single crossing instead of two previously. For the sake of clarity, three classes of horizontal wrap-around edges are distinguished: "special top" (only one edge), "normal" ($k-2$ edges) and "special bottom" (only one edge). This third improvement concerns only the "special top" horizontal wrap-around edge. See Figure 6; the "special top" edge is highlighted in green.

As a result, the non-wrap-around torus edges induce in total $(k-1)^2$ crossings, the leftmost vertical wrap-around edge $k-2$ crossings and the other $k-1$ vertical wrap-around edges $k-1$ crossings each. The "special top" horizontal wrap-around edge induces one single crossing, the $k-2$ "normal" horizontal wrap-around edges one crossing each and the "special bottom" horizontal wrap-around edge no crossing. Therefore, the drawing method described in this third improvement induces a total of $(k-1)^2+(k-2)+(k-1)(k-1)+1+(k-2)=2k^2-2k-1$ crossings. This

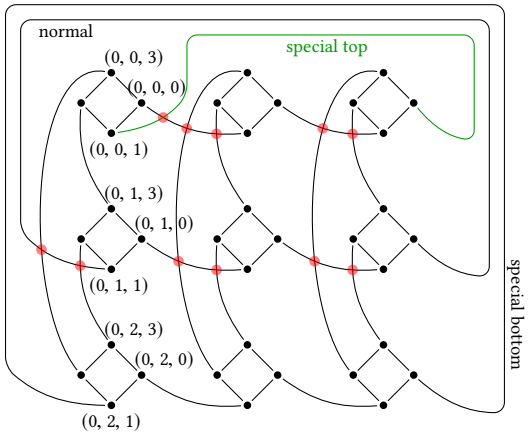


Figure 6: Third improvement of the TCC drawing method: the total number of crossings is further reduced from $2k^2-2k$ to $2k^2-2k-1$ (highlighted in red). This figure illustrates the case $k=3$, with thus a total of 11 crossings.

result is in accordance with the description of this improvement given at the beginning of this section. Figure 6 illustrates the case $k=3$, with thus a total of 11 crossings.

4.5 Fourth improvement

The fourth improvement concerns refining the drawing method of the topmost horizontal non-wrap-around torus edges so that they each induce at most two crossings instead of at most three previously. See Figure 7; these edges whose drawing has been refined are highlighted in green.

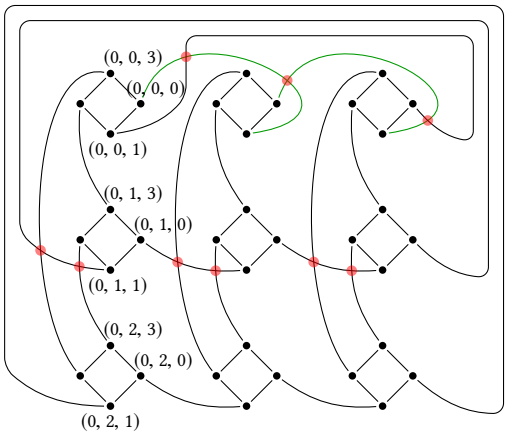


Figure 7: Fourth improvement of the TCC drawing method: the total number of crossings is further reduced from $2k^2-2k-1$ to $2k^2-2k-3$ (highlighted in red). This figure illustrates the case $k=3$, with thus a total of 9 crossings.

Hence, the non-wrap-around torus edges induce in total $(k-2)+(k-2)(k-1)$ crossings and the vertical wrap-around edges $k(k-2)$ crossings. The "special top" horizontal wrap-around edge induces

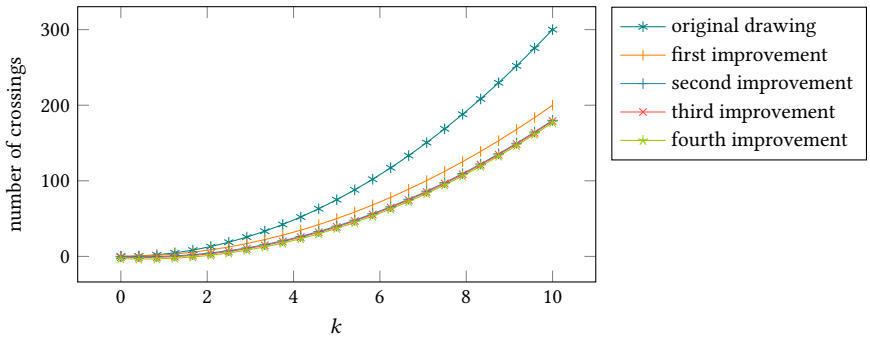


Figure 8: Plot of the successively obtained upper bounds on the crossing number of a TCC$(2, k)$.

$k-1$ crossings, the $k-2$ "normal" horizontal wrap-around edges one single crossing each and the "special bottom" horizontal wrap-around edge no crossing (these horizontal wrap-around edge classes are those of Section 6). Therefore, the drawing method described in this fourth improvement induces a total of $(k-2)+(k-2)(k-1)+k(k-2)+(k-1)+(k-2) = 2k^2-2k-3$ crossings. This result is in accordance with the description of this improvement given at the beginning of this section. Figure 7 illustrates the case $k=3$, with thus a total of 9 crossings.

5 RESULT DISCUSSION

In this section, we review the significance of the successive drawing method improvements described in Section 4 and challenge the obtained upper bound on $cr(\text{TCC}(2,k))$ with a computer experiment.

The first improvement reduces the total number of crossings in a TCC$(2,k)$ from $3k^2$ to $2k^2$. This is a significant result as the difference between these two upper bounds $\delta_1(k) = 3k^2-2k^2 = k^2$ diverges: we indeed have $\lim_{k\to+\infty}\delta_1(k) = +\infty$.

The second improvement reduces the total number of crossings in a TCC$(2,k)$ from $2k^2$ to $2k^2-2k$. This is once again a significant result as the difference between these two upper bounds $\delta_2(k) = 2k^2-(2k^2-2k) = 2k$ diverges: we again have $\lim_{k\to+\infty}\delta_2(k) = +\infty$.

Regarding the third improvement, it, as explained, concerns one single edge: while it previously induced two crossings, thanks to this improvement it now induces only one. This improvement is thus not as significant as the first and second ones as the difference between the previous upper bound and this one $\delta_3(k) = (2k^2-2k)-(2k^2-2k-1) = 1$ does not diverge – it is constant.

And, the fourth improvement reduces the total number of crossings in a TCC$(2,k)$ from $2k^2-2k-1$ to $2k^2-2k-3$, an improvement thus not as significant as the first two: the difference between the previous upper bound and this one $\delta_4(k) = (2k^2-2k-1)-(2k^2-2k-3) = 2$ does not diverge.

So as to easily compare the successively obtained upper bounds on the crossing number of a TCC$(2,k)$, we plot the corresponding functions of k the arity of the network in Figure 8.

Finally, in order to better assess the contribution of the proposed drawing method, we conducted a computer experiment whose purpose is to tentatively establish the crossing number of a TCC$(2,3)$ by successively refining automatically computed upper bounds. Concretely, this enables to compare the crossing number upper bound obtained theoretically to upper bounds obtained experimentally. This is a simulation experiment that is based on a program which implements a brute force approach to the crossing number problem as next described. The approach of this program is to trial numerous repartitions of graph vertices to positions of a two-dimensional grid and to calculate the induced number of crossings, thus producing an upper bound on the crossing number. Precisely, successive permutations of the grid positions are iteratively generated in lexicographical order and each serve as one arrangement of the sequence of graph vertices. Every possible edge pair – all the edge pairs have been generated in advance – is checked for the presence of a crossing in the current grid positioning of the graph vertices.

The experimental environment and conditions were as follows. Because the total number of possible vertex arrangements on the grid is prohibitive ($g^2!$, precisely, where g is the grid side, and thus g^2 grid positions, with obviously $g^2 \geq |\text{TCC}(n,k)| = 2nk^n$ required), the experiment was stopped after running for 90 minutes. Only the case of a TCC$(2,3)$ was considered as it is already computationally expensive. A 6×6 grid was chosen for vertex positioning as explained and the experiment was run on a computer equipped with an Intel Core i7-6700 CPU clocked at 3.40 GHz, 16 GB RAM and Windows 10 Home 64-bit.

The results are shown in Figure 9: for each vertex arrangement on the grid, the number of crossings is calculated and retained if it is smaller than the previously found upper bound. It can be seen that the graph drawings automatically generated with this computer program induce 102 as the minimum number of crossings. This is far from the proposed drawing method (only nine crossings), which thus tends to indicate the good performance of the proposal: a drawing method based on brute force achieves results after a running time of 90 minutes that are nowhere near the upper bound we formally established.

6 CONCLUSIONS

The torus-connected cycles topology has been proposed for the interconnection network of massively parallel systems. Torus-based supercomputers, like the Fujitsu Fugaku, world no. 1 as of May 2022, are in fact frequent nowadays. In this paper, we have established topological properties of 2-dimensional k-ary torus-connected cycles networks: it has been shown that a TCC$(2,k)$ is not symmetric

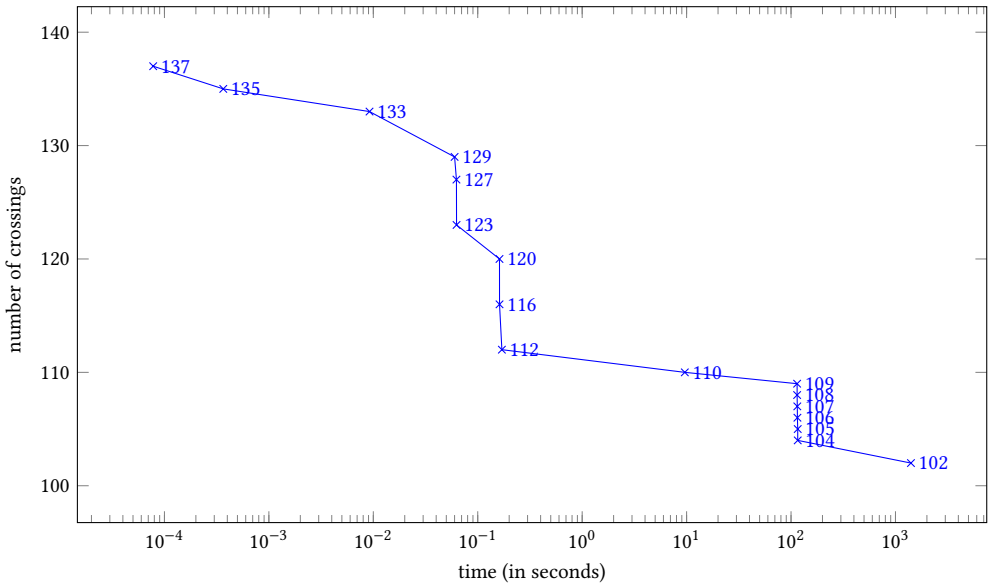


Figure 9: Successive approximations of the crossing number of a TCC$(2,3)$ **as calculated by a computer program with a brute force approach during 90 minutes. The best upper bound on the minimum number of crossings found for this graph is 102, thus well above our proposal.**

and that $2k^2-2k-3$ is an upper bound on its crossing number. These two difficult problems have important implications for algorithm design, such as routing algorithms, and circuit design, amongst others. The contribution of the proposal was further emphasised with a computer experiment run to approximate the crossing number of a TCC$(2,3)$: the upper bounds obtained empirically after a fully automatic 90-minute brute force run are significantly worse than the upper bound induced by our proposal.

Regarding future works, investigating whether the obtained upper bound is tight or it is possible to further reduce the number of crossings when drawing a TCC$(2,k)$ on the plane is one possible research direction. In addition, considering higher TCC dimensions (i.e. $n \geq 3$) is another interesting future work. As explained, this is a difficult problem, so another computer experiment might be useful to hint at the crossing number with empirical results.

ACKNOWLEDGMENTS

This research was partly supported by a Grant-in-Aid for Scientific Research (C) of the Japan Society for the Promotion of Science under grant no. 19K11887.

REFERENCES

[1] Yuichiro Ajima, Takahiro Kawashima, Takayuki Okamoto, Naoyuki Shida, Kouichi Hirai, Toshiyuki Shimizu, Shinya Hiramoto, Yoshiro Ikeda, Takahide Yoshikawa, Kenji Uchida, and Tomohiro Inoue. 2018. The Tofu Interconnect D. In *Proceedings of the IEEE International Conference on Cluster Computing (CLUSTER; Belfast, UK, 10–13 September)*. 646–654. https://doi.org/10.1109/CLUSTER.2018.00090

[2] Yuichiro Ajima, Shinji Sumimoto, and Toshiyuki Shimizu. 2009. Tofu: A 6D mesh/torus interconnect for exascale computers. *Computer* 42, 11 (2009), 36–40. https://doi.org/10.1109/MC.2009.370

[3] László Babai. 1995. Automorphism groups, isomorphism, reconstruction. In *Handbook of Combinatorics*, Ronald L. Graham, Martin Grötschel, and László Lovász (Eds.). Vol. 2. North-Holland, Amsterdam, The Netherlands.

[4] Norman Biggs. 1974. *Algebraic Graph Theory* (second ed.). Cambridge University Press, Cambridge, United Kingdom.

[5] Antoine Bossard and Keiichi Kaneko. 2013. Torus-Connected Cycles: an implementation-friendly topology for interconnection networks of massively parallel systems. In *Algorithms and Architectures for Parallel Processing (Lecture Notes in Computer Science, Vol. 8286)*, Rocco Aversa, Joanna Kołodziej, Jun Zhang, Flora Amato, and Giancarlo Fortino (Eds.). Springer International Publishing, Cham, 11–21. https://doi.org/10.1007/978-3-319-03889-6_2 Proceedings of the 13th International Conference on Algorithms and Architectures for Parallel Processing (ICA3PP; Vietri sul Mare, Italy, 18–20 December).

[6] Antoine Bossard and Keiichi Kaneko. 2015. A routing algorithm solving the container problem in a hypercube with bit constraint. *International Journal of Networked and Distributed Computing* 3 (2015), 202–213. Issue 4. https://doi.org/10.2991/ijndc.2015.3.4.1

[7] Antoine Bossard and Keiichi Kaneko. 2015. Torus–Connected Cycles: a simple and scalable topology for interconnection networks. *International Journal of Applied Mathematics and Computer Science* 25, 4 (2015), 723–735. https://doi.org/10.1515/amcs-2015-0052

[8] Gary Chartrand, Linda Lesniak, and Ping Zhang. 2015. *Graphs & digraphs* (sixth ed.). Chapman and Hall/CRC, Boca Raton, FL, USA.

[9] Dong Chen, Noel A. Eisley, Philip Heidelberger, Robert M. Senger, Yutaka Sugawara, Sameer Kumar, Valentina Salapura, David L. Satterfield, Burkhard Steinmacher-Burow, and Jeffrey J. Parker. 2011. The IBM Blue Gene/Q interconnection network and message unit. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC; Seattle, WA, USA, 12–18 November)*. 1–10.

[10] Reinhard Diestel. 2016. *Graph Theory* (fifth ed.). Springer-Verlag, Heidelberg, Germany.

[11] José Duato, Sudhakar Yalamanchili, and Lionel Ni. 2003. *Interconnection Networks – An Engineering Approach*. Morgan Kaufmann, San Francisco, CA, USA. Revised printing.

[12] Charles L. Seitz. 1985. The Cosmic Cube. *Commun. ACM* 28, 1 (1985), 22–33. https://doi.org/10.1145/2465.2467

[13] TOP500. 2021. Still waiting for Exascale: Japan's Fugaku outperforms all competition once again. https://top500.org/news/still-waiting-exascale-japans-fugaku-outperforms-all-competition-once-again/. Last accessed May 2022.

[14] Jian Zhang, Chunbao Zhou, Yangang Wang, Lili Ju, Qiang Du, Xuebin Chi, Dongsheng Xu, Dexun Chen, Yong Liu, and Zhao Liu. 2016. Extreme-scale phase field simulations of coarsening dynamics on the Sunway TaihuLight supercomputer. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC; Salt Lake City, UT, USA, 13–18 November)*. 34–45. https://doi.org/10.1109/SC.2016.3